

Homework 7

```
const express = require('express')
const axios = require('axios')

const app = express()
app.use(express.json())

app.post('/api/orders', async (req, res) => {
  const order = req.body
  const payment = await axios.post('http://payments-
service/pay', { orderId: order.id, amount: order.total })
  const shipping = await axios.post('http://shipping-
service/ship', { orderId: order.id, address: order.address })
  res.json({
    orderId: order.id,
    paymentStatus: payment.data.status,
    shipmentId: shipping.data.shipmentId
  })
})

app.listen(3000, () => console.log('API Gateway listening on
3000'))
```

The API Gateway pattern is a Service-Based system built around microservices and API management. It allows clients to make requests to a system using a predefined set of entry points. This enables scalability, allowing for easy growth and high flexibility by supporting many features, and allows easy additions. However, maintainability can be painful for clients if alterations to existing API interfaces are required. This is a good fit for what this use case is: an interface to aggregate user requests for orders for payment and shipping. This allows for many things to happen on the back end with only one client call required. This can also be easily expanded to also handle different kinds of authentication or verification with no change to the clients.

As a client, you would only see one endpoint: `POST /api/orders`. This is how any requesting client would request a new order. This would be made accessible to the client through the API Gateway created from the lines: `const app = express(), app.use(express.json())`. From there, multiple other requests are made in order to provide feedback to the client. This is implemented in two parts: the first part is the API Gateway posting and awaiting payment and shipping information from another service, then the “express” application will relay the response of the order ID, payment, and shipping information. All this builds into one API that a client can utilize to perform many actions.

Now, with this implementation, there are a few concerns I have. My concerns are primarily with how the await functions are handled. These are a sequential waiting function with no timeout. This could potentially hang for a very long time, when it would make sense from the customer perspective to get an error message if something were to not respond within a reasonable amount of time. Another potential optimization would be to use a “`Promise.all(insert the posts here)`” instead of the await function, so that each server response is independent of the others. However, I see one critique of this would be that a seller would likely want the payment to be processed before shipping is handled, so this one may be opted for not implementing.